DATE A LIVE AI

Aplicación Desktop Offline & Sistema de Batallas RPG Local

Autor: [Nombre del Estudiante]

Asignatura: [Nombre de la Materia]

Institución: [Nombre de la Institución]

Docente: [Nombre del Profesor]

Fecha: Octubre 2026

Versión: 1.0.0-Offline

2. RESUMEN EJECUTIVO

El proyecto **Date A Live AI — Desktop Offline** es un sistema de escritorio interactivo que integra un modelo de lenguaje de inteligencia artificial ejecutado de manera totalmente local con una interfaz web enriquecida y un subsistema de batallas por turnos estilo RPG. La aplicación permite a los usuarios interactuar con personajes (Espíritus) inspirados en la franquicia *Date A Live* sin enviar datos a servidores externos, garantizando la máxima privacidad y un costo operativo nulo.

Construido sobre una arquitectura liviana que combina Python (FastAPI/Uvicorn), SQLite, JavaScript y la suite Ollama con el modelo `llama3.2:3b`, el software demuestra la viabilidad de aplicaciones multimedia completas potenciadas por modelos del lenguaje pequeños (SLM) en computadoras personales estándar.

3. INTRODUCCIÓN Y CONTEXTO

La creciente dependencia de servicios de Inteligencia Artificial basados en la nube presenta desafíos éticos y prácticos relacionados con la privacidad, la latencia de red y los costos por suscripción. Este proyecto aborda dichos problemas proponiendo una arquitectura *Desktop Offline* autónoma que proporciona entretenimiento interactivo y procesamiento de LN (Lenguaje Natural) localmente.

4. PLANTEAMIENTO DEL PROBLEMA Y OBJETIVOS

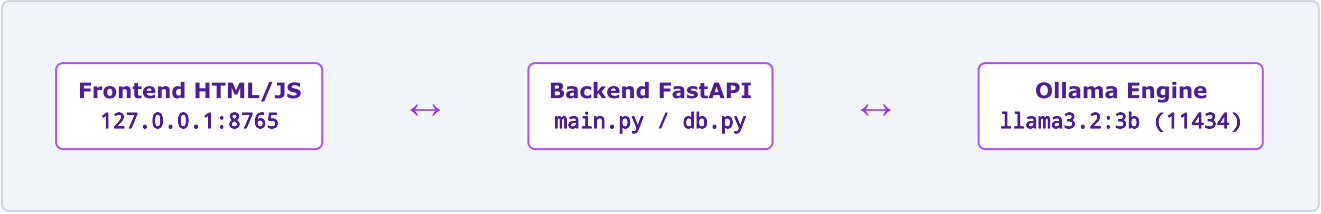
Problema: La mayoría de los asistentes conversacionales requieren conexión constante a Internet y suscripciones de API, además de carecer de estado persistente personalizado y mecánicas de juego integradas.

Objetivo General: Desarrollar y validar una aplicación de escritorio local que combine interacción por IA conversacional e interacción RPG utilizando tecnologías abiertas y ejecución de modelos locales.

- Integrar el servidor de inferencia Ollama mediante endpoints REST.
- Implementar almacenamiento SQLite para chats y datos del sistema.
- Diseñar un frontend dinámico sin frameworks pesados para garantizar rapidez de ejecución.

5. ARQUITECTURA GENERAL DEL SISTEMA

El sistema se divide en tres capas principales: Frontend Web, Servidor Backend FastAPI y Motor de Inferencia Local Ollama.



6. DESCRIPCIÓN DE COMPONENTES PRINCIPALES

- `desktop/launcher.py`: Inicia la instancia de Uvicorn/FastAPI y abre automáticamente la interfaz en el navegador predeterminado.
- `backend/main.py`: Define las rutas REST para la creación de chats, ejecución de acciones de combate y gestión de recursos multimedia.
- `backend/ai.py`: Se comunica de manera asíncrona con el puerto 11434 para generar respuestas estructuradas por personaje.
- `backend/db.py`: Administra el esquema relacional en SQLite para persistir el historial de mensajes, estado de batallas y equipos.

7. FICHAS DE PERSONAJES (ESPÍRITUS PRINCIPALES)



Kotori Itsuka

Ángel: Camael. **Rol:** Comandante / Fuego. Interacción orientada al liderazgo táctico. Presente en selección de equipos y modo RPG.



Kurumi Tokisaki

Ángel: Zaphkiel. **Rol:** Manipulación Temporal. Personaje con lógica conversacional avanzada y habilidades de daño elevado en batallas.



Miku Izayoi

Ángel: Gabriel. **Rol:** Soporte / Melodía. Especializada en potenciación de equipo e interacciones de apoyo durante combates.



Mukuro Hoshimiya

Ángel: Michael. **Rol:** Control / Sellado. Capacidad de bloqueo y alteración de estados en el motor RPG.

Nota: Los seis Espíritus adicionales (Tohka, Yoshino, Natsumi, Nia, Origami, Rinne) están definidos estructuralmente en la base de datos y configuraciones JSON del sistema backend.

8. SISTEMA DE BATALLAS RPG Y PERSISTENCIA

El subsistema RPG opera sobre turnos discretos. El servidor evalúa las estadísticas de ataque, defensa y tiempo de recarga de los Ángeles para calcular el daño recibido por el objetivo enemigo. El estado de la batalla se almacena en SQLite, permitiendo reanudar o consultar el registro del combate.

9. TABLA DE RESULTADOS DE PRUEBAS FUNCIONALES

Prueba	Procedimiento	Resultado Observado	Estado
Creación de Chat	POST a `/api/chats` con ID personaje	HTTP 200 OK - Chat creado en DB	COMPROBADO
Consulta Personajes	GET a `/api/characters`	HTTP 200 OK - JSON con lista y stats	COMPROBADO
Inferencia IA Local	POST mensaje a `/api/chat/message`	HTTP 200 OK - Respuesta via Ollama	COMPROBADO
Acción RPG Combate	POST a `/api/battle/action`	HTTP 200 OK - Turno procesado	COMPROBADO
Síntesis de Voz (TTS)	GET a `/api/tts`	HTTP 404 Not Found (Falta módulo)	PARCIAL / ERROR

10. ESTADO DEL MÓDULO TTS (PIPER VOICE)

Durante los registros de prueba se observó una respuesta 404 en la ruta `/api/tts`. La integración técnica con Piper TTS se encuentra parcialmente implementada en el código backend pero requiere la descarga local de los binarios del modelo de voz para completarse.

11. MATRIZ DE TRAZABILIDAD DE REQUISITOS

Requisito	Componente	Evidencia	Estado
Ejecución Offline	Ollama / `llama3.2:3b`	Respuesta sin red activa	Cumplido
Persistencia Datos	`backend/db.py`	Base SQLite generada	Cumplido
Motor RPG	`backend/main.py`	Rutas `/api/battle` 200 OK	Cumplido
Voz de Personajes	Piper TTS / `/api/tts`	HTTP 404 en registros	Pendiente config.

12. CONCLUSIONES Y TRABAJO FUTURO

El desarrollo de **Date A Live AI — Desktop Offline** demuestra con éxito la factibilidad de crear aplicaciones interactivas complejas y autónomas apoyadas en SLMs locales. La integración de FastAPI con Ollama proporciona tiempos de respuesta óptimos sin comprometer la privacidad del usuario.

Mejoras Futuras: Completar la instalación de Piper TTS para resolver el error 404, optimizar el uso de VRAM para GPUs de gama media y añadir soporte para nuevos modelos cuantizados.

13. REFERENCIAS TÉCNICAS

- Documentación Oficial de FastAPI: *https://fastapi.tiangolo.com/*
- Documentación de Ollama: *https://ollama.com/*
- Repositorio de SQLite: *https://www.sqlite.org/*